



Asignatura: Paradigmas y Lenguajes de Programación

Trabajo Práctico Integrador: Sistema de Semáforos Inteligentes y Análisis Comparativo de Paradigmas.

Comisión: 1

Grupo: 16

Integrantes:

Arredondo Vera Araceli Estefany - Usuario GitHub: Nayuu-vv

Aquino Anabela Del Valle - Usuario GitHub : aquinoanabela

Galarza Juan Pablo - Usuario GitHub : jpgala12-gif

Gonzalez Oviedo Bruno Fabricio - Usuario GitHub : FabricioGonzalez99

Quiñones Fernanda - Usuario GitHub : fernandaeq / [ferr-eq](#)

Fecha: 16 de Junio 2026.

Introducción

El trabajo tiene como objetivo desarrollar un sistema inteligente de semáforos utilizando Common Lisp, aplicando conceptos del paradigma funcional para modelar transiciones de estados, temporización automática, auditoría de eventos y análisis de ciclos semafóricos.

Además, se incorporó una librería externa mediante Quicklisp para extender las capacidades del sistema y se realizó un análisis comparativo de paradigmas según lo solicitado en la Fase 3 con el lenguaje asignado por la cátedra OCaml.

Fundamentos del diseño funcional adoptado

Inmutabilidad

Una de las principales restricciones del proyecto fue la prohibición de utilizar variables globales mutables (`defparameter`, `defvar`) y operadores de modificación de estado (`setq`, `setf`).

Por este motivo, el sistema fue diseñado para que toda la información necesaria circule a través de parámetros y valores de retorno, evitando la existencia de estados compartidos modificables.

Por ejemplo, la función `transicion` (requerimiento 1) determina el siguiente estado del semáforo únicamente a partir de los parámetros recibidos, sin almacenar información en variables globales.

Uso predominante de funciones puras

La mayoría de las funciones implementadas son puras porque siempre producen el mismo resultado para los mismos argumentos, no modifican estructuras existentes y no generan efectos secundarios.

Ejemplos:

- `transicion`
- `timer_con_intermitencia`
- `duracion-ciclo`
- `recomendacion-ciclo`
- `ciclos-por-tiempo`
- `porcentaje`

La única excepción es `sistema-auditoria` que debe escribir información en un archivo externo.

Composición funcional

Se priorizó la reutilización de funciones para mejorar la modularidad y facilitar el mantenimiento.

Por ejemplo en:

- ciclos-por-tiempo usa duracion-ciclo
- informe-distribucion-temporal usa duracion-ciclo y porcentaje.
- sistema-auditoria usa obtener-fecha-formateada.

Ausencia de estructuras iterativas imperativas

Debido a las restricciones del proyecto cuando fue necesario procesar conjuntos de datos se recurrió a herramientas funcionales como: (every ...) para validar listas.

Un ejemplo es la función duracion-ciclo, donde se uso every con una función (lambda) para verificar que todos los tiempos ingresados sean números enteros no negativos.

Validación funcional de entradas

Se incorporaron validaciones explícitas mediante: integerp,member y every , con el objetivo de garantizar que las funciones reciban únicamente datos válidos antes de realizar los cálculos correspondientes.

Casos de Prueba

Se realizaron casos de prueba para cada requerimiento. Los casos alternativos representan caminos válidos diferentes al principal, mientras que los casos de error verifican las validaciones implementadas en cada requerimiento.

REQUERIMIENTO 1: Estados de Transición

Caso normal:

(transicion 'en-verde 'verde-intermitente)

Resultado esperado:

(EN-VERDE CAMBIAR-A-VERDE-INTERMITENTE)

Camino alternativo:

(transicion 'en-verde 'rojo)

Resultado esperado:

(EN-VERDE ACCION-POR-DEFECTO)

Caso de error:

(transicion 'azul 'verde)

Resultado esperado:

NIL

REQUERIMIENTO 2 : Temporizador Automático

Caso normal:

(timer_con_intermitencia 50)

Resultado esperado:

EN-ROJO

Camino alternativo:

(timer_con_intermitencia 214)

Resultado esperado:

VERDE-INTERMITENTE

Otro camino alternativo:

(timer_con_intermitencia 223)

Resultado esperado:

AMARILLO-INTERMITENTE

Caso de error:

(timer_con_intermitencia -10)

Resultado esperado:

ERROR-TIEMPO-INVALIDO

(timer_con_intermitencia 'hola')

Resultado esperado:

ERROR-TIEMPO-INVALIDO

REQUERIMIENTO 3 : Sistema de Auditoría

Caso normal:

(sistema-auditoria)

Resultado esperado:

Se agrega una nueva línea al archivo
"informe-ejecucion-semaforo.txt"
con fecha, hora y transición registrada.

Camino alternativo:

Ejecutar varias veces:

(sistema-auditoria)

(sistema-auditoria)

Resultado esperado:

Las nuevas líneas se agregan al final del archivo sin sobrescribir las anteriores.

Caso de error:

Un posible error sería no disponer de permisos de escritura sobre el archivo.

REQUERIMIENTO 4 : Análisis de Ciclos Semafóricos

Función duracion-ciclo :

Caso normal:

(duracion-ciclo)

Resultado esperado:

225

Camino alternativo:

(duracion-ciclo 100 5 10 5 120 5)

Resultado esperado:

245

Caso de error:

(duracion-ciclo 90 3 6 -1 120 3)

Resultado esperado:

ERROR-TIEMPO-INVALIDO

Función recomendacion-ciclo :

(recomendacion-ciclo 120)

Resultado esperado:

CICLO-OPTIMO

Camino alternativo:

(recomendacion-ciclo 20)

Resultado esperado:

CICLO-DEMASIADO-CORTO

(recomendacion-ciclo 200)

Resultado esperado:

CICLO-DEMASIADO-LARGO

Caso de error:

(recomendacion-ciclo "hola")

Resultado esperado:

ERROR-TIEMPO-INVALIDO

REQUERIMIENTO 5 : Planificación Temporal

Caso normal:

(ciclos-por-tiempo 15)

Resultado esperado:

4

(15 minutos = 900 segundos,
 $900 / 225 = 4$ ciclos completos)

Camino alternativo:

(ciclos-por-tiempo 1)

Resultado esperado:

0

(No alcanza para completar un ciclo)

Caso de error:

(ciclos-por-tiempo -5)

Resultado esperado:

VALOR-INVALIDO

(ciclos-por-tiempo 'hola)

Resultado esperado:

VALOR-INVALIDO

REQUERIMIENTO 6 : Informe de Distribución Temporal

Caso normal:

(informe-distribucion-temporal)

Resultado esperado:

(ROJO "40.00%"

ROJO-INTERMITENTE "1.33%"

AMARILLO "2.67%"

AMARILLO-INTERMITENTE "1.33%"

VERDE "53.33%"

VERDE-INTERMITENTE "1.33%")

Camino alternativo:

No posee caminos alternativos ya que no recibe parámetros y utiliza valores constantes definidos por el sistema.

Caso de error:

No tiene casos de error directos porque no recibe datos externos. Su correcto funcionamiento depende de las funciones auxiliares **duracion-ciclo** y **porcentaje**, cuyos casos de error son verificados de forma independiente.

Función porcentaje :

Caso normal:

(porcentaje 90 225)

Resultado esperado:

40.0

Camino alternativo:

(porcentaje 3 225)

Resultado esperado:

1.3333333

Caso de error:

(porcentaje -5 225)

Resultado esperado:

VALOR-INVALIDO

(porcentaje 'hola 225)

Resultado esperado:

VALOR-INVALIDO

Bitácora de Depuración:

ERROR 1:

```
(defun transicion(color-actual cambiar-a) ;; Cambiar y hacer la validacion del ciclo Verde-amarillo amarillo-rojo
  (let ((colores_incluidos (list 'en-rojo 'en-amarillo 'en-verde))(colores_cambio (list 'rojo 'amarillo 'verde)))
    (cond
      ((null (member color-actual colores_incluidos)) "El valor de color actual es incorrecto") ;; Agregamos fu
      ((and (member cambiar-a colores_cambio)(list color-actual 'cambiar-a- cambiar-a))) ;; La funcion member
      (t (list color-actual 'accion-por-defecto))) ;; Ver donde se verifica el dato si aca o en alguna entrada
    )
  )
)
```

Función transicion

El Error Breve: * Sintaxis: Todo el bloque de la transición quedó metido dentro de un solo gran paréntesis, haciendo que Lisp interprete el retorno como parte de la condición.

Semántica: Usaba member de forma genérica, lo que permitía transiciones imposibles (ej. pasar de verde a rojo directo). Además, mezclaba tipos de retorno devolviendo un String en caso de error y una Lista por defecto.

Cómo lo solucionamos: Se desglosó la máquina de estados paso a paso comparando átomo por átomo con eql. Si el color actual es inválido, ahora devuelve un valor consistente (nil).

ERROR 2:

```
57 ;; =====
58 ;; FUNCION: Duracion del ciclo
59 ;; NATURALEZA: Pura
60 ;; ESTRATEGIA: Calculo aritmetico
61 ;; IMPACTO EN MEMORIA: No destructiva
62 ;; =====
63 (defun duracion-ciclo (&optional (seg-rojo 90)(int-rojo 3)(seg-amarillo 6)(int-amarillo 3)(seg-verde 120)(int-verde 3)) ;; &optional utiliza valores por defecto
64   (unless
65     (every #'(lambda (x)
66               (and (integerp x)
67                    (>= x 0)))
68           (list seg-rojo seg-amarillo seg-verde
69               int-rojo int-amarillo int-verde))
70     'error-tiempo-invalido)
71     (+ seg-rojo seg-amarillo seg-verde int-rojo int-amarillo int-verde)
72   )
73 )
74
```

Función duracion-ciclo

El Error Breve: Error de flujo crítico. La operación de suma (+) quedó fuera de la estructura del unless. Si la validación fallaba, el programa entraba al unless, registraba el símbolo de error, pero inmediatamente seguía de largo y ejecutaba la suma de todas formas con los datos inválidos.

Cómo lo solucionamos: Se reemplazó el unless por un if. Esto divide el flujo en dos ramas completamente excluyentes: si los datos son correctos se suma, de lo contrario se corta la ejecución devolviendo el error.

ERROR 3:

```
157 /
158
159 ;; =====
160 ;; FUNCION: Ciclos por tiempo
161 ;; NATURALEZA: Pura
162 ;; ESTRATEGIA: Composición funcional, calculo aritmetico y validación condicional
163 ;; IMPACTO EN MEMORIA: No destructiva
164 ;; =====
165 (defun ciclos-por-tiempo (minutos)
166   (if (< minutos 0) 'valor-invalido
167       (nth-value 0(truncate (* minutos 60)(duracion-ciclo))))
168   )
169 )
170
171 ;; =====
```

Función ciclos-por-tiempo

El Error Breve: Falta de validación del tipo de dato; si se pasaba algo que no fuera un número (ej. un string o un símbolo), el operador < rompía la ejecución. Además, al corregir esto, la lógica condicional del if se invirtió.

Cómo lo solucionamos: Se aplicó una validación defensiva con and e integerp por cortocircuito. Si el argumento no es un entero mayor o igual a cero, se salta la operación matemática y devuelve 'valor-invalido.

ERROR 4:

```
(defun sistema_auditoria(tiempo)
  (let* ((color-actual (timer tiempo))
        (cambiar-a (cond
                     ((eql color-actual 'en-verde) 'amarillo)
                     ((eql color-actual 'en-amarillo) 'rojo)
                     (t 'verde)
                     )
        ))
    (transicion color-actual cambiar-a)
    (format t "Tiempo ~A: la luz ha cambiado de ~A a ~A" tiempo color-actual cambiar-a)
  )
)
```

Función sistema_auditoria

El Error Breve: * Efecto secundario: La función format t devuelve siempre nil. Al estar en la última línea, "pisaba" el resultado real de la transición y la función de auditoría no retornaba nada útil.

Lógica: El ciclo de luces original era demasiado simple e ignoraba los estados intermitentes. También dependía de un parámetro estático de tiempo.

Cómo lo solucionamos: Se reestructuró la máquina de estados completa usando let*. El tiempo ahora se calcula de manera dinámica y autónoma sincronizándolo con el Tiempo Unix (get-universal-time menos la constante de segundos correspondientes).

ERROR 5

```
;;=====
=====
REQUERIMIENTO 6: Informe de Distribución
Temporal ;; FUNCIÓN: Porcentaje ;;
NATURALEZA: Pura ;; ESTRATEGIA: Calculo
aritmético ;; IMPACTO: No destructivo ;;
=====
=====

(defun porcentaje(tiempo-color total-ciclo) ;
Total del ciclo rojo 90 + amarillo 6 + verde
120 = 216 seg (* (/ tiempo-color (float total-
ciclo)) 100) )
```

Función porcentaje

El Error Breve: * Sintaxis fatídica: El programador escribió un comentario con punto y coma (;) pero metió la fórmula matemática y los paréntesis de cierre en esa misma línea. Lisp ignoró todo lo que seguía al ;;, dejando la función vacía y rota.

Dominio: El cálculo omitía los 3 segundos de los intervalos intermitentes. Carecía además de control para evitar errores críticos como la división por cero.

Cómo lo solucionamos: Se separó físicamente el código del comentario. Se añadió un bloque if con validación defensiva estricta para asegurar que los datos sean enteros

positivos y que el total del ciclo contemple de forma real las variables intermitentes (225 seg en total).

Fase 2: Autonomía y Ecosistema

Justificación de la Librería Seleccionada en la Fase 2:

Se eligió la librería local-time para mejorar la utilidad de los registros generados por el Sistema de Auditoría con fines forenses. En la versión original, los eventos se almacenaban utilizando tiempo Unix (Epoch), un formato adecuado para el procesamiento por computadora pero poco práctico para el análisis humano.

Mediante la integración de local-time, cada cambio de estado del semáforo queda registrado con una fecha y hora legible, por ejemplo: [2026-05-16 14:30:00]. Esto facilita la revisión de los eventos, el análisis posterior ante posibles fallas o comportamientos anómalos del sistema y la reconstrucción cronológica de los hechos.

En caso de incidentes, accidentes o reportes de funcionamiento incorrecto, los registros permiten identificar con mayor precisión cuándo ocurrió cada cambio de estado del semáforo, proporcionando información útil para tareas de investigación, auditoría y análisis forense.

La incorporación de esta librería permite mejorar la interpretación de la información registrada sin modificar la lógica principal de funcionamiento del semáforo, manteniendo el comportamiento original del sistema y optimizando las tareas de auditoría y seguimiento para los analistas forenses.

Fase 3: Estudio Comparativo OCaml

OCaml (Objective Caml) es un lenguaje de programación de propósito general que combina características de la programación funcional, imperativa y orientada a objetos. Fue desarrollado en Francia por investigadores del instituto INRIA y se destaca por ayudar a detectar errores antes de ejecutar los programas.

Una de sus características más importantes es la inferencia de tipos que permite al compilador deducir automáticamente qué tipo de datos utiliza cada función sin que el programador tenga que indicarlo explícitamente. También cuenta con herramientas como Pattern Matching que facilitan el trabajo con distintos casos o valores de una manera más ordenada y segura.

Gracias a estas características OCaml es utilizado en proyectos donde la confiabilidad del software es muy importante y donde un error puede generar grandes pérdidas económicas o problemas de funcionamiento.

Áreas de aplicación y empresas que utilizan OCaml

OCaml se utiliza principalmente en áreas donde se requiere software seguro y confiable.

Finanzas : En el sector financiero se emplea para desarrollar sistemas que procesan operaciones de compra y venta de activos de manera rápida y segura.

Jane Street: es una empresa de trading cuantitativo que opera las 24 horas del día, los 7 días de la semana, en todo el mundo que utiliza OCaml en gran parte de sus sistemas.

Bloomberg: ha utilizado OCaml en algunas herramientas relacionadas con el análisis de datos financieros.

Análisis y verificación de software : OCaml también se utiliza para desarrollar herramientas que ayudan a detectar errores en programas antes de que lleguen a los usuarios.

Meta (Facebook): utilizó OCaml en proyectos como Infer y Flow, herramientas destinadas a analizar código y detectar posibles errores.

Microsoft: ha empleado OCaml en proyectos de investigación relacionados con la verificación de software.

Blockchain y criptografía : Debido a su enfoque en la seguridad y la confiabilidad, también es utilizado en tecnologías relacionadas con blockchain.

Tezos: una plataforma de blockchain desarrollada en gran parte utilizando OCaml.

Experiencia de estudio y utilización del lenguaje :

La experiencia de estudiar y utilizar OCaml nos permitió conocer una forma diferente de programar respecto a Common Lisp. Al principio nos costó adaptarnos a su sintaxis y a la forma en que maneja los tipos de datos pero con la práctica pudimos comprender mejor sus ventajas. Uno de los aspectos que más nos llamó la atención fue que muchos errores son detectados por el compilador antes de ejecutar el programa y esto nos ayudó a entender la importancia de los lenguajes con tipado estático y cómo pueden contribuir a desarrollar software más confiable. También nos resultó interesante el uso de Pattern Matching que permite organizar la lógica de los programas de manera clara y ordenada. En general consideramos que trabajar con OCaml fue una experiencia enriquecedora porque nos permitió comparar dos enfoques diferentes dentro de la programación funcional y ampliar nuestra visión sobre el desarrollo de software.

Respuestas teóricas a los ejes comparativos :

1. Al igual que Haskell, OCaml usa tipos estáticos pero cuenta con Inferencia de Tipos. ¿Tuvieron que declarar explícitamente de qué tipo eran las funciones o el compilador lo dedujo solo? Muestren cómo lo interpreta el entorno.

No tuvimos que declarar explícitamente el tipo de las funciones gracias a la inferencia de tipos con la que cuenta OCaml, ya que el compilador deduce automáticamente los tipos de los parámetros y del retorno al analizar el cuerpo de la función.

El compilador deduce esto basándose en las operaciones y funciones internas que se aplican sobre los argumentos.

Por ejemplo:

En la **función transicion** : se utiliza `List.mem` para verificar si `color_actual` pertenece a la lista `colores_incluidos`. Como la lista contiene cadenas de texto, el compilador deduce que `color_actual` debe ser de tipo `string` y al observar que la función devuelve valores de la forma `Some ( ... )` o `None` deduce que el tipo de retorno corresponde a un valor opcional (`option`).

En la **función timer_con_intermitencia**: Al utilizar el operador `mod` y la comparación `< 0`, el compilador identifica inmediatamente que el parámetro `tiempo_unix` debe ser un entero (`int`), ya que esas operaciones son exclusivas para ese tipo de dato. Esto hace innecesaria una validación manual de tipo en tiempo de ejecución (como el `integerp` que usábamos en Lisp) ya que si se intentara pasar un flotante u otro tipo el programa directamente arrojaría un error durante la compilación.

Interpretación del entorno (Firmas obtenidas en las capturas):

Como se puede observar en las imágenes adjuntas de las pruebas que se encuentran al final de las respuestas, el entorno interactivo de OCaml calculó y expuso las firmas de la siguiente manera:

```
val transicion_con_intermitencia : string -> string -> (string * string) option = <fun>
```

```
val timer_con_intermitencia : int -> string = <fun>
```

Las flechas (`->`) demuestran cómo el sistema infirió el flujo completo de los tipos de entrada hacia el tipo del valor resultante de manera 100% automatizada.

2. OCaml es un lenguaje "orientado a expresiones". ¿Cómo cambia esto la estructura de control de flujos (como el uso de `match...with`) comparado con los condicionales de Lisp?

Decir que OCaml está orientado a expresiones, significa que cada bloque de código o estructura que escribimos debe producir obligatoriamente un valor de retorno. No existen instrucciones vacías que solo ejecuten acciones sin retornar nada.

Por ejemplo: en nuestra función `timer_con_intermitencia`:

```
if tiempo_unix < 0 then
  "error-tiempo-invalido"
else
  ...
```

La expresión `if` devuelve un valor de tipo `string` en cualquiera de sus ramas. Esto obliga a que tanto la parte `then` como la parte `else` sean compatibles en tipo.

En Common Lisp utilizamos principalmente la estructura `cond`, que evalúa una secuencia de condiciones booleanas independientes hasta encontrar una verdadera. En cambio, OCaml utiliza `match...with`, una estructura basada en Pattern Matching (comparación de patrones), que permite analizar directamente la forma o el valor de los datos.

En nuestro caso utilizamos cláusulas `when` para agregar condiciones adicionales sobre los patrones:

```
match residuo with
| r when r < 90 -> "en-rojo"
| r when r < 93 -> "rojo-intermitente"
```

Sin el uso de `when`, `match` únicamente podría buscar coincidencias exactas de valores y no permitiría evaluar rangos numéricos como los requeridos por el temporizador del semáforo.

Al ser un lenguaje fuertemente tipado y orientado a expresiones, OCaml impone ciertas reglas durante la construcción del flujo de control:

Unificación de tipos: todas las ramas de un `match` deben devolver el mismo tipo de dato. En nuestro caso, todas las alternativas retornan cadenas de texto (`string`).

Exhaustividad: el compilador exige que todos los casos posibles estén cubiertos. Para ello utilizamos el comodín (`_`), que cumple una función similar al caso por defecto (`t ...`) utilizado en `cond` de Common Lisp.

A diferencia de Common Lisp donde muchas validaciones se realizan durante la ejecución del programa OCaml detecta una gran cantidad de errores potenciales durante la compilación gracias a su sistema de tipos estático y al control de exhaustividad de los patrones.

Imágenes de las pruebas:

```
# let transicion_con_intermitencia color_actual cambiar_a =
  (*let ... in es la variante de let para la declaracion de variables locales*)
  let colores_incluidos = ["en_rojo"; "en_amarillo"; "en_verde";
    "rojo_intermitente"; "amarillo_intermitente"; "verde_intermitente"] in

  (*list.mem funciona como member, buscando en la lista el valor que se desea
  encontrar*)
  if List.mem color_actual colores_incluidos then
    (*equivalencia de cond en ocaml*)
    match (color_actual, cambiar_a) with
      (*Some se utiliza para entregar un valor, en este caso some esta asignando a
      color actual el valor correspondiente*)
      | ("en_verde", "verde_intermitente") -> Some (color_actual, "cambiar-a-verde-
intermitente")
      | ("verde_intermitente", "amarillo") -> Some (color_actual, "cambiar-a-
amarillo")
      | ("en_amarillo", "amarillo_intermitente") -> Some (color_actual, "cambiar-a-
amarillo-intermitente")
      | ("amarillo_intermitente", "rojo") -> Some (color_actual, "cambiar-a-rojo")
      | ("en_rojo", "rojo_intermitente") -> Some (color_actual, "cambiar-a-rojo-
intermitente")
      (*ultimo paso del cond, si no es ninguno de los anteriores cambia por
defecto*)
      | _ -> Some (color_actual, "cambio-por-defecto")
    else
      None
  ;;

val transicion_con_intermitencia :
string -> string -> (string * string) option = <fun>
```

```
None
;;
val transicion_con_intermitencia :
string -> string -> (string * string) option = <fun>
# let prueba = transicion_con_intermitencia "en_rojo" "rojo_intermitente" ;;
val prueba : (string * string) option =
Some ("en_rojo", "cambiar-a-rojo-intermitente")
# let error = transicion_con_intermitencia "azul" "amarillo" ;;
val error : (string * string) option = None
#
```

```
# (*funcion que mide el tiempo de cada luz del semaforo*)
let timer_con_intermitencia tiempo_unix =
  (*en caso de ser un numero negativo*)
  if tiempo_unix < 0 then
    "error-tiempo-invalido"
  else
    (*resto de la division, variable local*)
    let residuo = tiempo_unix mod 225 in

    (*aquí, a r se le asigna el valor del residuo, y luego se compara con los
    valores*)
    match residuo with
      | r when r < 90 -> "en-rojo"
      | r when r < 93 -> "rojo-intermitente"
      | r when r < 213 -> "en-verde"
      | r when r < 216 -> "verde-intermitente"
      | r when r < 222 -> "en-amarillo"
      | _ -> "amarillo-intermitente"
    ;;

val timer_con_intermitencia : int -> string = <fun>
# let prueba_error = timer_con_intermitencia 45;;
val prueba_error : string = "en-rojo"
# let prueba_error = timer_con_intermitencia (-4);;
val prueba_error : string = "error-tiempo-invalido"
#
```

Bibliografía:

Teoría de Cátedra de Paradigmas y Lenguajes de Programación (2026).

[Tutorial para principiantes: Git y GitHub](#)

[git - Básico.pdf](#)

[Introduction | OCaml Programming | Chapter 1 Video 1](#)

<https://ocaml.org/>